

S1 Text. Event decoding, label formulas and extraction queries.

This text gives what the main text summarizes: which logs were read, how each field was decoded, how every label was computed, and which properties of the data the sensitivity analyses listed in the Discussion would have to address. Constants are those of the analysis pipeline; the repository and commit that hold them are named in the Data availability statement.

1. Source

All records are rows of the public BigQuery table `bigquery-public-data.goog_blockchain_arbitrum_one_us.logs` (Arbitrum One, chain identifier 42161). Each row is one event log with its block timestamp, block number, transaction hash, log index, emitting contract (`address`), topics and data. Queries were run month by month with a dry-run cost estimate, filtered by block timestamp (UTC), by topic 0 and, for ERC-20 logs, by a list of addresses that contains the seed cohort; only records with a seed wallet at one end enter the graphs. The queries are listed in Section 6.

2. ERC-20 approvals

Selection and fields. Topic 0 equals `0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925`, the keccak-256 hash of `Approval(address,address,uint256)`. The query requires at least three topics. Topic 1 is the owner and topic 2 the spender (the last 20 bytes of each 32-byte word, lower-cased); the emitting contract is the token; the data field is the allowance in the token’s base units, cast to a double-precision float with no division by the token’s decimals. An EIP-2612 `permit` emits the same event and is therefore included. ERC-721 contracts emit an event with the same signature but put the token identifier in a fourth topic and leave the data empty; such a log decodes to zero and is removed with the other zero records. Exclusion is thus by decoded value, not by topic count; a four-topic log with non-empty data would be read as an ERC-20 approval.

Latest allowance. For each (token, owner, spender) triple the record with the largest (block number, log index) up to the freeze is kept. If its value is zero the triple is removed. An edge $u \rightarrow v$ of the allowance graph exists when at least one token has a positive latest value for the pair; its EndorseRank weight sums $\sigma(\Delta t^*)V(a^*)$ over those tokens, with Δt^* the age of the kept record at the freeze. Approvals made before 1 December 2025 are not observed.

Known properties not quantified in this study. (a) Many ERC-20 implementations, among them those built on the OpenZeppelin library before its version 5, also emit `Approval` from inside `transferFrom` when they reduce a finite allowance. For such tokens the kept record may be the remaining allowance after the spender’s last use and not a new authorization: its age then dates the last spend, an exact allowance spent in full ends at zero and drops out, and an old authorization spent during a label window can appear as a new approval pair. The share of `Approval` logs emitted in the same transaction as a `Transfer` by the same token with the same owner and spender would measure this; it was not computed. (b) Contracts that manage allowances of their own, such as Uniswap’s Permit2, record those inner allowances in events with a different signature, which the extraction does not read. Owners approve such a contract through an ordinary ERC-20 approval, so it appears as one spender, and the addresses it authorizes in turn are not seen. Its share of edges was not computed.

3. ERC-20 transfers

Topic 0 equals `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`, the keccak-256 hash of `Transfer(address,address,uint256)`. Topic 1 is the sender, topic 2 the recipient and the data

field the amount in base units. Zero-value transfers and the zero-decoding ERC-721 logs are removed. Mint and burn logs (zero address at one end) are kept when a seed wallet is at the other end. Self-transfers stay in the events table but are excluded from transfer in-degree and from the new-sender label. GMX V2 settles collateral and profits in ERC-20 tokens, so payouts from GMX contracts to traders are ordinary transfers and appear in the pre-freeze transfer graph; their share of edges was not computed.

4. GMX V2 position decreases

Selection. GMX V2 emits its position events through one EventEmitter contract, `0xC8ee91A54287DB53897056e12D9819156D3822Fb` on Arbitrum One. The query keeps logs of that contract with topic 0 equal to `0x137a44067c8961cd7e1d876f4754a5a3a75989b4552f1843fc69c3b372def160` (the `EventLog1` event) and topic 1 equal to the keccak-256 hash of the string `PositionDecrease` (the event name, indexed as a string). These logs are not filtered by account, so the cohort can be selected from them.

Decoding. Each log is decoded with the EventEmitter ABI. The account is the `account` entry of the address items (topic 2 as a fallback); the profit or loss is the `basePnlUsd` entry of the signed integer items; the order type is the `orderType` entry of the unsigned integer items; `sizeDeltaUsd` and `isLong` are read but enter no label. A log without `basePnlUsd` is skipped.

Close and liquidation. Every `PositionDecrease` log is one close, whether it reduces a position partly or closes it. A close is a liquidation if its order type equals 7, the value of `Liquidation` in GMX V2's order-type enumeration, or if the event's `msgSender` is the configured liquidation handler `0xaf157Eb8e2398A8E1Fc1dA929974652b9ba9BC25`. No other decrease, including one executed by auto-deleveraging, is flagged. `basePnlUsd` is used as recorded: fees and other amounts that GMX reports in separate events are not subtracted, and only its sign enters every label except the realized gain.

5. Label formulas

Traders. For a seed wallet i let n_i be its closes in the label window, L_i the liquidated ones, W_i the closes that were not liquidations and had `basePnlUsd` > 0 , and B_i the closes that were not liquidations and had `basePnlUsd` < 0 . Labels are defined only when $n_i \geq 3$:

$$\begin{aligned} \text{liquidation-free close rate} &= 1 - L_i/n_i, & \text{no-liquidation flag} &= \mathbf{1}[L_i = 0], \\ \text{profitable closes} &= W_i, & \text{realized gain} &= \sum_{\text{closes}} \max(\text{basePnlUsd}, 0), \\ \text{profitable share} &= W_i/n_i, & \text{non-loss share} &= 1 - B_i/n_i. \end{aligned}$$

Two consequences follow from these definitions. Because partial decreases count as closes, a trader who reduces positions in many steps has more closes per liquidation and a higher liquidation-free close rate than one who closes in a single step; a position-level rate (liquidated positions over positions closed) was not computed. Because a liquidated close has neither W nor B status, it counts against the profitable share but in favor of the non-loss share; a trader whose every close was liquidated has a non-loss share of one. The non-loss share therefore rises with liquidations by construction.

Spenders. New approval pairs of spender v : the number of distinct owners u that emit a positive `Approval` for v on any token inside the label window while the pair (u, v) held no positive latest allowance at the freeze. New transfer senders of v : the number of distinct addresses $u \neq v$ that send v a transfer inside the label window and sent it none between 1 December 2025 and the freeze.

6. Account types

For the 1,335 spenders of the spring cohort, `eth_getCode` was called on an Arbitrum One node at block 510,408,687 (end of September 2026), after both label windows. Non-empty code was classified as a

contract, empty code as an externally owned account (EOA), and code beginning with the EIP-7702 delegation designator `0xef0100` as an EOA that delegates to contract code; one of the 161 EOAs is of this kind. Because the state read is that of September and not of the freeze dates, an account deployed after a freeze (for example a counterfactually deployed smart account) would be classified as a contract. Traders that were not spring spenders were not classified.

7. Queries

Parameters are bound at run time: `@start_ts` and `@end_ts` delimit one month, `@wallet_addresses` is the lower-cased address list, and the topic and contract constants are those given above.

```
-- ERC-20 Approval logs touching the seed cohort
SELECT block_timestamp, block_number, transaction_hash, log_index,
       address AS token_address,
       topics[SAFE_OFFSET(1)] AS owner_topic,
       topics[SAFE_OFFSET(2)] AS spender_topic,
       data AS value_data
FROM `bigquery-public-data.google_blockchain_arbitrum_one_us.logs`
WHERE block_timestamp >= TIMESTAMP(@start_ts)
      AND block_timestamp <  TIMESTAMP(@end_ts)
      AND ARRAY_LENGTH(topics) >= 3
      AND topics[SAFE_OFFSET(0)] = @approval_topic0
      AND (LOWER(CONCAT('0x', SUBSTR(topics[SAFE_OFFSET(1)], 27))) IN UNNEST(@wallet_addresses)
           OR LOWER(CONCAT('0x', SUBSTR(topics[SAFE_OFFSET(2)], 27))) IN UNNEST(@wallet_addresses));

-- ERC-20 Transfer logs: the same query with @transfer_topic0,
-- topics 1 and 2 read as sender and recipient.

-- GMX V2 PositionDecrease logs (all accounts)
SELECT block_timestamp, block_number, transaction_hash, log_index,
       address, topics, data
FROM `bigquery-public-data.google_blockchain_arbitrum_one_us.logs`
WHERE block_timestamp >= TIMESTAMP(@start_ts)
      AND block_timestamp <  TIMESTAMP(@end_ts)
      AND LOWER(address) = LOWER(@event_emitter)
      AND ARRAY_LENGTH(topics) >= 3
      AND topics[SAFE_OFFSET(0)] = @event_log1_topic0
      AND topics[SAFE_OFFSET(1)] = @position_decrease_hash;
```

The run dates of the extractions are recorded in the pipeline’s extraction manifests; the June–August logs were extracted on 1 October 2026.